

Twitter Clone

Full-Stack Architecture Blueprint

[Stack](#) · [Architecture](#) · [Features](#) · [Chat](#) · [Security](#) · [Roadmap](#)

React 19

Node.js

PostgreSQL

Redis

Socket.IO

Docker

Prepared as a comprehensive technical reference for building a production-grade Twitter (X) clone web application from scratch, covering the full development lifecycle.

Document	Twitter Clone — Full-Stack Architecture Blueprint
Version	1.0
Scope	Web Application (PWA-ready)
Stack	React 19 · Node.js · PostgreSQL · Redis · Socket.IO · Docker

Table of Contents

1. Technology Stack
2. Client–Server Architecture
3. Database Schema Overview
4. Core Features
5. Real-Time Chat & Messaging System
6. Security Vulnerabilities & Mitigations
7. API Design Principles
8. Execution Roadmap — 8 Phases
9. DevOps & Infrastructure
10. Summary & Next Steps

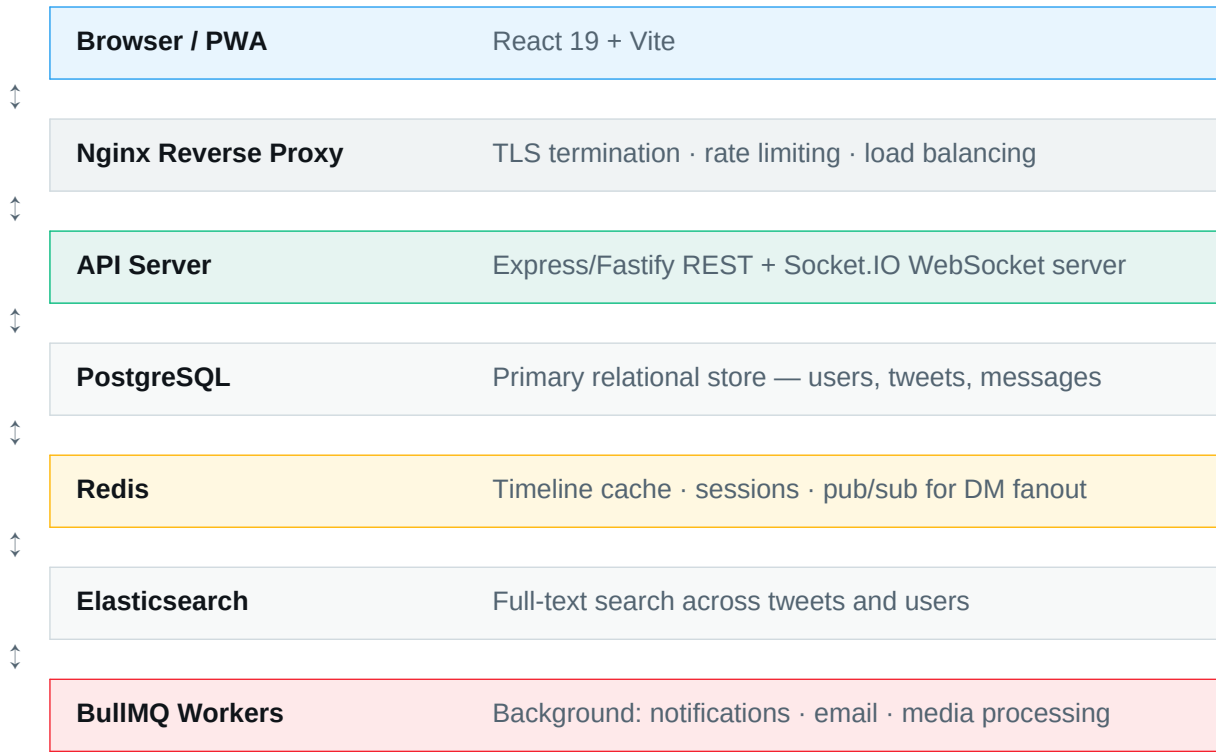
1. Technology Stack

The stack is chosen for developer productivity, type safety, real-time capability, and horizontal scalability — all requirements of a social media platform at scale.

Layer	Technology	Purpose
Frontend	React 19 + Vite	Component UI, fast HMR dev builds
Language	TypeScript (strict)	End-to-end type safety
Styling	Tailwind CSS + shadcn/ui	Utility-first, consistent design system
State	Zustand + TanStack Query	Global state + server-state caching
Routing	React Router v7	Client-side routing with loaders
Real-time client	Socket.IO client	WebSocket DMs and notifications
Backend	Node.js + Express/Fastify	REST API + WebSocket server
ORM	Prisma	Type-safe DB queries, migrations
Primary DB	PostgreSQL 16	Relational data, ACID transactions
Cache / Pub-Sub	Redis 7	Sessions, timelines, DM fanout
Search	Elasticsearch 8	Full-text tweet and user search
Auth	Passport.js + JWT	Local + OAuth2 (Google, GitHub)
Queue	BullMQ	Notifications, email, media workers
Media	Cloudinary / S3	Image/video storage and CDN delivery
Containerization	Docker + Compose	Reproducible dev and prod environments
Proxy	Nginx	Reverse proxy, TLS, rate limiting
CI/CD	GitHub Actions	Lint, test, build, deploy pipeline
Monitoring	Prometheus + Grafana	Metrics, dashboards, alerting
Error tracking	Sentry	Runtime error capture and alerts

2. Client–Server Architecture

The application follows a layered client-server model with a clear separation of concerns at each tier. All communication between the browser and backend passes through Nginx.



Key architecture decisions

Auth transport	JWT access token (15 min TTL) + httpOnly Refresh cookie (7 days). Rotate on each use.
Media uploads	Presigned S3 URL — browser uploads directly to S3, bypassing the API server entirely.
Feed strategy	Fan-out-on-write for accounts with ≤10k followers. Pull-on-read for high-follower accounts (celebrities).
Real-time	Socket.IO rooms per conversation + Redis pub/sub so messages fan out across multiple API server instances.
Caching	Redis for home timeline, trending topics, and user profiles. TTL: 60 s (timeline) to 5 min (profiles).
Search indexing	Elasticsearch updated asynchronously via BullMQ job on tweet creation or deletion.

3. Database Schema Overview

PostgreSQL is the source of truth. Below are the primary tables and their key columns.

Table	Key Columns
users	id, username, email, password_hash, bio, avatar_url, header_url, is_verified, created_at
sessions	id, user_id (FK), refresh_token_hash, device_info, expires_at, created_at
tweets	id, author_id (FK), content, media_urls[], parent_id (FK), quote_id (FK), type (tweet reply retweet quote), c
likes	user_id (FK), tweet_id (FK), created_at — composite PK
retweets	user_id (FK), tweet_id (FK), created_at — composite PK
bookmarks	user_id (FK), tweet_id (FK), created_at
follows	follower_id (FK), following_id (FK), created_at — composite PK
conversations	id, is_group, name, created_by (FK), created_at
conversation_members	conversation_id (FK), user_id (FK), joined_at, last_read_at
messages	id, conversation_id (FK), sender_id (FK), content, media_url, deleted_at, created_at
notifications	id, recipient_id (FK), actor_id (FK), type (like reply follow mention retweet), entity_id, read_at, created_at
hashtags	id, tag, tweet_count, trending_score, updated_at
tweet_hashtags	tweet_id (FK), hashtag_id (FK) — junction table
media	id, uploader_id (FK), url, cdn_url, type (image video gif), width, height, alt_text, created_at

4. Core Features

Authentication

- Email/password signup and login with bcrypt hashing
- OAuth2 login via Google and GitHub (Passport.js)
- Email verification on signup (tokenized link)
- Password reset via time-limited email token
- Multi-device session management with revocation
- Two-factor authentication (TOTP) — optional

Tweets & Interactions

- Create tweets up to 280 characters with media attachments
- Reply, quote-tweet, and retweet functionality
- Like and bookmark tweets
- Delete own tweets (soft delete + cascade)
- Thread / long-form post support
- Polls with configurable duration (1h – 7 days)
- Sensitive content warning toggle

Social Graph

- Follow and unfollow users
- Home feed — tweets from followed accounts (fan-out)
- For-you algorithmic feed (engagement signals)
- Mute and block users
- Profile pages with bio, header image, follower counts
- Who-to-follow suggestions (collaborative filtering)
- Verified account badge support

Search & Explore

- Full-text tweet search powered by Elasticsearch
- User search by handle or display name
- Hashtag explore and topic pages
- Trending topics (Redis sorted sets, 24h window)
- Search filters: media only, people, date range
- Recent searches history (localStorage)

Notifications

- In-app notification centre with badge count
- Real-time push notifications via WebSocket
- Email digest notifications (BullMQ + Resend)
- Events: likes, replies, follows, mentions, retweets
- Mark individual or all notifications as read
- Notification preferences per event type

Media

- Image upload: JPEG, PNG, GIF (max 5 MB)
- Video upload with background transcoding (ffmpeg)
- Up to 4 images or 1 video per tweet
- Alt text for accessibility compliance
- EXIF metadata stripped for privacy
- CDN delivery via Cloudinary (global edge caching)
- Sensitive media flag and blur overlay

5. Real-Time Chat & Messaging System

Direct Messages (DMs) use Socket.IO over WSS for persistent bidirectional communication. Redis pub/sub enables message fanout across multiple API server instances.

Architecture

Protocol	Socket.IO over WSS — persistent bidirectional connection per client
Rooms	Each conversation has a UUID room. Users join their rooms on socket connect after auth verification.
Pub/Sub	Redis Pub/Sub for multi-server fanout — any server instance can receive a message and publish to all subscribers.
Persistence	Messages stored in PostgreSQL messages table. <code>last_read_at</code> tracked per member for unread counts.
Delivery	Undelivered messages (recipient offline) are queued and delivered on next socket connect.
Encryption	TLS in transit (WSS). Sensitive fields encrypted at rest. Passwords never stored in messages.
Presence	Redis key per user with 30s TTL, refreshed by heartbeat. Expiry = user went offline.

Chat features

- 1-to-1 private conversations
- Group conversations (up to 50 members)
- Image and GIF sharing inside DMs
- Emoji reactions on individual messages
- Message deletion (soft delete — shown as "message removed")
- Typing indicators (debounced, not persisted)
- Read receipts — "seen" timestamp per member
- Online / last active status display
- Conversation search (client-side over loaded messages)
- Unread message count badge per conversation
- Mute conversation (suppress notifications)
- Leave group conversation

Socket event schema

Event	Direction	Description
-------	-----------	-------------

message:send	Client → Server	Send a new message. Server validates, persists, then emits message:new to room.
message:new	Server → Client	Broadcast to all room members (online). Includes full message payload.
message:delivered	Server → Sender	ACK that server received and persisted the message.
message:delete	Client → Server	Soft-delete a message. Emits message:deleted to room.
user:typing	Client → Server	Debounced typing event. Server broadcasts to room excluding sender.
conversation:join	Client → Server	Join socket room. Server verifies membership before allowing join.
presence:update	Server → Room	Broadcast user online/offline status change to conversation room.
message:react	Client → Server	Add emoji reaction. Server persists and broadcasts to room.

Chat security controls

- Socket connection requires a valid JWT in the handshake Authorization header
- Server verifies conversation membership before allowing any room join
- Rate limit: 30 messages per minute per user (Redis sliding window)
- All message content sanitized server-side to prevent XSS
- Block/mute controls prevent unwanted messages
- Spam detection worker (BullMQ) flags high-velocity senders
- No anonymous DMs — both users must have accounts
- Report message functionality with admin moderation queue

6. Security Vulnerabilities & Mitigations

All OWASP Top 10 attack vectors are addressed. Below is each risk with its concrete mitigation.

! **Authentication & Session Hijacking**

Short-lived JWTs (15 min) + httpOnly Refresh token cookies. Rotate refresh token on each use. Invalidate all sessions on password change. CSRF tokens on every state-changing request. Log and alert on anomalous IPs.

! **SQL Injection**

Prisma ORM with parameterized queries — zero raw string interpolation. When raw SQL is unavoidable, use tagged template literals with bound parameters exclusively.

! **XSS — Cross-Site Scripting**

DOMPurify sanitization on all user content server-side. React escapes JSX output by default. Strict Content Security Policy (CSP) header. Never use dangerouslySetInnerHTML. Validate and sanitize all URLs in tweet content.

! **CSRF — Cross-Site Request Forgery**

Double-submit CSRF token pattern on all non-GET mutations. SameSite=Strict cookies. Validate Origin and Referer headers. Socket.IO authenticated via JWT in handshake header, never via cookie.

! **Rate Limiting & DDoS**

Nginx rate limiting at the ingress layer. Per-IP + per-user sliding-window limits in Redis. BullMQ throttling on background jobs. Exponential backoff enforced on auth endpoints. Cloudflare WAF in production.

! **Broken Access Control (IDOR)**

Every resource endpoint verifies ownership server-side before every operation. Users cannot delete or edit other users tweets. DM room membership verified on every socket join. Never trust client-supplied user IDs.

! **Malicious File Uploads**

Validate MIME type and magic bytes server-side (not just Content-Type header). Strip all EXIF metadata for user privacy. Virus scan via ClamAV worker before CDN delivery. Images re-encoded (canvas API) to strip hidden payloads. Media served from an isolated CDN domain.

! **Sensitive Data Exposure**

Passwords hashed with bcrypt (cost factor 12+). Tokens stored as hashes only. HTTPS enforced everywhere with HSTS. Database credentials rotated quarterly. No secrets in source code or git history — use environment variables and secrets managers.

! **Security Headers (HTTP)**

Helmet.js enforces: HSTS, X-Frame-Options: DENY, X-Content-Type-Options: nosniff, Referrer-Policy: strict-origin-when-cross-origin, Permissions-Policy, strict Content Security Policy. Force HTTPS in all environments.

! **Dependency Vulnerabilities**

Dependabot or Renovate for automated dependency PRs. npm audit in CI pipeline — fail build on high severity. Only use packages with active maintainers and security policies. Lock file committed to source control.

7. API Design Principles

Base URL	/api/v1/ — versioned from day one to allow non-breaking evolution
Style	RESTful — standard HTTP verbs, resource-based URLs, consistent status codes
Auth	Bearer JWT in Authorization header for API calls. httpOnly cookie for refresh token endpoint.
Pagination	Cursor-based pagination (not offset) for feeds and search to avoid gaps on new inserts.
Rate limits	200 req/15 min for authenticated users. 30 req/15 min for unauthenticated. 429 with Retry-After.
Error format	{ "error": { "code": "TWEET_NOT_FOUND", "message": "...", "status": 404 } }
Validation	Zod schemas server-side on all request bodies. Return 422 with field-level errors on failure.
CORS	Strict origin whitelist. Only the production frontend domain allowed in production.
Compression	Gzip/Brotli compression via Nginx for all JSON and static asset responses.

Core API endpoints

Method	Endpoint	Description
POST	/auth/register	Register new user
POST	/auth/login	Login, receive JWT + set refresh cookie
POST	/auth/refresh	Exchange refresh cookie for new JWT
POST	/auth/logout	Revoke refresh token
GET	/users/:username	Get public profile
PUT	/users/me	Update own profile
GET	/tweets/home	Home timeline feed (cursor paginated)
GET	/tweets/for-you	Algorithmic feed
POST	/tweets	Create tweet / reply / quote
DELETE	/tweets/:id	Delete own tweet
POST	/tweets/:id/like	Like a tweet
DELETE	/tweets/:id/like	Unlike a tweet
POST	/tweets/:id/retweet	Retweet

POST	/follows/:userId	Follow a user
DELETE	/follows/:userId	Unfollow a user
GET	/search/tweets	Search tweets (?q=&filter=)
GET	/search/users	Search users
GET	/trending	Get trending hashtags
GET	/conversations	List user conversations
POST	/conversations	Create new DM conversation
GET	/conversations/:id/messages	Get messages (cursor paginated)
GET	/notifications	Get user notifications
PUT	/notifications/read-all	Mark all notifications as read

8. Execution Roadmap — 8 Phases

Phases are ordered so each one builds on the previous. Complete Phase 1 before touching Phase 2.

1	Project Scaffold & DevOps Setup Monorepo with Turborepo. Docker Compose: Postgres, Redis, API server, React client. Nginx reverse proxy config. GitHub Actions CI pipeline (lint, typecheck, test). ESLint + Prettier + Husky pre-commit hooks. Environment variable management with dotenv-safe.
2	Database Schema & Auth System Prisma schema for all tables (users, tweets, follows, messages, notifications). Auth endpoints: register, login, logout, refresh token, OAuth2 (Passport.js). JWT access token + httpOnly refresh cookie. Email verification flow. Password reset via time-limited tokens.
3	Tweet CRUD & Social Graph Create / read / delete tweets, replies, retweets, quote tweets. Like and bookmark endpoints. Follow / unfollow API. Home timeline feed with Redis caching and BullMQ fan-out workers. Cursor-based pagination throughout.
4	Media Uploads Presigned S3 URL generation endpoint. Client uploads directly to S3 — API server not in the upload path. Image validation (magic bytes) + EXIF stripping worker. Cloudinary integration for CDN delivery. Video transcoding queue using BullMQ + ffmpeg.
5	Real-Time Chat (DMs) Socket.IO server with JWT authentication in handshake. Redis pub/sub for multi-instance fanout. Conversation and message models. 1-to-1 and group DMs. Typing indicators, read receipts, online/offline presence. Unread count badges updated via WebSocket.
6	Notifications In-app notification centre with real-time WebSocket delivery. Notification model in Postgres (like, reply, follow, mention, retweet). BullMQ email worker using Resend or SendGrid. Web Push API for browser push notifications. Unread badge on bell icon. Per-event-type preference settings.
7	Search & Explore Elasticsearch index for tweets, updated via BullMQ on create/delete. Full-text search API endpoint with filters. Hashtag aggregation and dedicated explore page. Trending topics using Redis sorted sets with a 24-hour rolling window. User search by handle and display name.

Polish, Security Hardening & Deployment

Helmet.js security headers. Full rate limiting audit (Nginx + Redis). OWASP Top 10 checklist review.

8

Load testing with k6 (target: 500 RPS). Sentry integration for error tracking. Prometheus metrics endpoint + Grafana dashboards. Production deployment to AWS / Railway / Render with zero-downtime deploys.

9. DevOps & Infrastructure

Containerization	Each service runs in its own Docker container. Docker Compose for local dev. Kubernetes (or ECS) for production.
Nginx	Terminates TLS. Routes /api/* to the Node server, /* to the React SPA. Enforces rate limits at the ingress.
CI pipeline	On every PR: ESLint, TypeScript typecheck, Jest unit tests, Prisma migration dry-run, Docker build. On merge to main: deploy to staging.
Secrets	No secrets in git. Dev: .env files (gitignored). Production: AWS Secrets Manager or Doppler.
Database backups	Automated daily Postgres snapshots to S3 with 30-day retention. Point-in-time recovery enabled.
Scaling strategy	API server: horizontal (stateless, session in Redis). Postgres: read replicas for feed queries. Redis: Redis Cluster for pub/sub at scale.
Observability	Prometheus scrapes /metrics. Grafana dashboards for RPS, latency p99, DB connections, queue depth. Alerts via PagerDuty or Slack.
CDN	Static assets (React bundle, images) served from Cloudfront or Cloudinary CDN. API server not involved in media delivery.

10. Summary & Next Steps

This document covers the complete technical blueprint required to build a production-grade Twitter clone. The architecture prioritises type safety, security-by-default, real-time capability, and the ability to scale horizontally as user count grows.

What this blueprint gives you

- A fully specified tech stack with rationale for each choice
- A layered client-server architecture with clear separation of concerns
- A normalised relational database schema covering all features
- A secure real-time chat system built on Socket.IO and Redis
- Mitigations for every OWASP Top 10 vulnerability
- A versioned REST API with cursor pagination and Zod validation
- An 8-phase execution roadmap ordered by dependency
- A production-ready DevOps setup with CI/CD, monitoring, and backups

Recommended starting point

Phase 1 first	Get Docker Compose, Nginx, and the monorepo scaffold running before writing any feature code.
Auth before features	A solid auth system (Phase 2) is the foundation everything else depends on. Do not skip.
Build with tests	Write Jest unit tests for auth and tweet logic from day one. Add Playwright e2e tests in Phase 8.
Security as you go	Apply Helmet, rate limiting, and Prisma parameterized queries from the start — not as an afterthought.
Deploy early	Get a staging environment running by Phase 3 so you can test real network conditions throughout.

This blueprint is a living document. Update it as the system evolves — architecture decisions made early have compounding effects.